# 《会学习的红绿灯》作品说明文档

Oct 11, 2026 · 作者：欧阳卓元（深圳市龙华区和平实验小学 四年级三班）

## 作品概览

《会学习的红绿灯》是一个会自己学会指挥十字路口的 AI 红绿灯：在完全相同的车流下，它让车辆平均等待时间比传统定时红绿灯减少约 60%。

| 项目 | 内容 |
| --- | --- |
| 作品名称 | 会学习的红绿灯 · 龙华智慧路口 AI 交通指挥官 |
| 作者 | 欧阳卓元 |
| 学校 | 深圳市龙华区和平实验小学 |
| 班级 | 四年级三班 |
| 参赛组别 | Python 小学组 |
| 编程语言 | Python 3（只用标准库，0 个第三方库） |
| 核心算法 | 从零手写的 Q 学习（强化学习）+ 经验回放 |
| 运行方式 | 电脑：海龟编辑器 / IDLE 打开 `main.py` 直接运行；网页 / 手机：浏览器打开在线演示 |
| 在线演示 | zongruichd-rd.github.io/bisai2 |
| 源代码 | github.com/ZONGRUICHD-Rd/bisai2 |
| 代码规模 | 约 2000 行 Python，9 个自动测试 |

## 一、设计意图

红绿灯应该看路况做决定，而不是机械地到点就切 —— 这是我做这个作品的出发点。

**身边的问题。** 上学路上常常看到：一边排了长长一串车，另一边一辆车都没有，可红灯还是要亮满时间。传统定时红绿灯不看路况，空等浪费时间，怠速的汽车还在不停排放尾气。

**龙华的特点。** 很多龙华居民早上往南去福田、南山上班，晚上再回来，早晚高峰的主要车流方向正好相反。一个固定的时间表，很难同时照顾好早高峰、晚高峰和深夜。

**为什么用强化学习。** 什么路况该切换、什么时候该保持，很难人工写成规则。强化学习不需要人告诉答案：AI 自己“试一试、看结果、总结经验”，就像人学骑自行车一样。

**我的目标。**

1. 用 Python 搭建一个十字路口交通仿真，车辆会跟车、会停车、不会闯红灯。
2. 从零手写 Q 学习算法，让 AI 控制红绿灯。
3. 把 AI 和传统定时灯放在完全相同的车流下公平比赛，用数据说话。
4. 让评委和同学不用安装任何东西，用浏览器就能体验。

## 二、作品功能

画面左边是传统定时红绿灯，右边是 AI 红绿灯，两个路口迎来一模一样的车流；右侧面板实时对比成绩、显示 AI 的“大脑”，下方曲线记录等待时间的变化。




| 功能 | 怎么用 | 说明 |
| --- | --- | --- |
| 双路口公平对比 | 打开就有 | 两个路口共用一个随机来车源，每辆车同一时刻、同一方向、同样速度到达（控制变量法） |
| 实时对比卡 | 右上角 | 平均等待、正在排队、怠速碳排放、通过车辆、救护车等待时间 |
| 智能一天 | 默认场景 | 时钟从 6:40 开始流动，早高峰南下车多、晚高峰北上车多、深夜车少 |
| 切换场景 | C 键 / 按钮 | 智能一天、早高峰、晚高峰、平峰、深夜 |

| 功能 | 怎么用 | 说明 |
| --- | --- | --- |
| AI 大脑可视化 | 右侧中间 | Q 表热力图：橙色 = 想切换，蓝色 = 想保持，“？”= 没遇到过，白框 = 现在的路况 |
| AI 心声 | Q 表下方 | 把每次决定翻译成人话，如“东西向有 6 辆车在等，南北向只剩 1 辆 → 切换” |
| 救护车优先 | A 键 / 按钮 | AI 路口立即为救护车开绿灯，定时路口只能干等 |
| 极速训练 | T 键 / 按钮 | AI 在看不见的虚拟路口里“做梦”练习 2 小时车流，几秒就完成 |
| 清空大脑 | R 键 / 按钮 | AI 忘掉一切从零开始，亲眼看 Q 表从一片“？”长出规律 |
| 学霸大脑 | L 键 / 按钮 | 载入训练好的大脑（已学习约 2 万次） |
| 环保账 | 对比卡底部 | 按“怠速 1 秒 ≈ 0.39 克 $CO_2$”统计，并换算成“相当于几棵树一天的吸收量” |
| 三种运行方式 | — | 电脑窗口、网页、手机全屏（竖屏自动转横屏），用的是同一份 Python 代码 |

清空大脑后 Q 表是一片“？”；极速训练之后，表里长出了规律 —— 红灯方向车多、绿灯方向车少时切换，反之保持。






手机上点“全屏演示”，竖着拿也会自动横过来，右侧是大号触屏按钮；电脑版用 tkinter 窗口运行，画面和网页版一模一样。






## 三、制作元器件与材料

本作品是纯软件作品，不需要硬件；没有使用任何第三方库或现成的 AI 库，Q 学习的每一行都是自己写的。

| 类别 | 材料 | 用途 |
| --- | --- | --- |
| 编程语言 | Python 3（3.6 及以上） | 全部功能 |
| 编辑器 | 海龟编辑器 / IDLE / VS Code | 编写和运行代码 |
| 标准库 | `tkinter` | 电脑版画面 |
| 标准库 | `random`、`math`、`json`、`collections` | 随机车流、物理计算、保存 AI 大脑、统计 |
| 标准库 | `unittest` | 9 个自动测试 |
| 网页运行 | Pyodide | 把 Python 解释器搬进浏览器，网页版直接运行同一份 Python 代码 |
| 发布 | GitHub Pages | 评委无需安装即可在线体验 |

**源代码文件清单**

| 文件 | 作用 |
| --- | --- |
| `main.py` | 电脑版入口，用 tkinter 把画面画出来 |
| `train.py` | 命令行训练和考试脚本 |
| `smartlight/config.py` | 所有可调参数（车速、黄灯时长、学习率……） |
| `smartlight/traffic.py` | 交通仿真：车辆、车道、信号灯、路口 |
| `smartlight/brain.py` | AI 大脑：Q 学习 + 经验回放 |
| `smartlight/controllers.py` | 定时控制器、AI 控制器、安全规则层 |
| `smartlight/scenario.py` | 一天中的车流变化 |
| `smartlight/world.py` | 对比实验场、极速训练、考试 |
| `smartlight/render.py` | 把世界“翻译”成绘图指令 |
| `smartlight/ui.py` | 按钮、键盘、浮层 |
| `smartlight/web_bridge.py` | 网页版和 JavaScript 对接 |
| `smartlight/pretrained_brain.json` | 训练好的“学霸大脑” |
| `tests/test_smartlight.py` | 自动测试 |

## 四、技术实现过程

作品分为五层：交通仿真 → AI 大脑 → 控制器（含安全规则）→ 对比实验场 → 画面，每层只管自己的事，改一层不会影响别的层。

### 1. 交通仿真

- 每辆车只用两个数描述：车头走了多远 `s`，当前速度 `v`。
- **跟车模型**：先找出前方“必须停下的位置”（前车车尾，或红灯时的停止线），再算出在这段距离内能安全刹住的最高速度 v = √(2 × 刹车减速度 × 距离)。车速永远不超过它，所以**不会追尾、不会闯红灯**。
- **时间车距**：行驶时和前车保持 0.45 秒的距离，开得越快离得越远。一条车道每秒约能通过 1 辆车，接近真实路口。
- **黄灯两难**：黄灯时能舒服停下就停，停不下就通过，和真实司机一样。
- **信号灯状态机**：绿 → 黄（2 秒）→ 全红（1 秒）→ 另一方向绿。

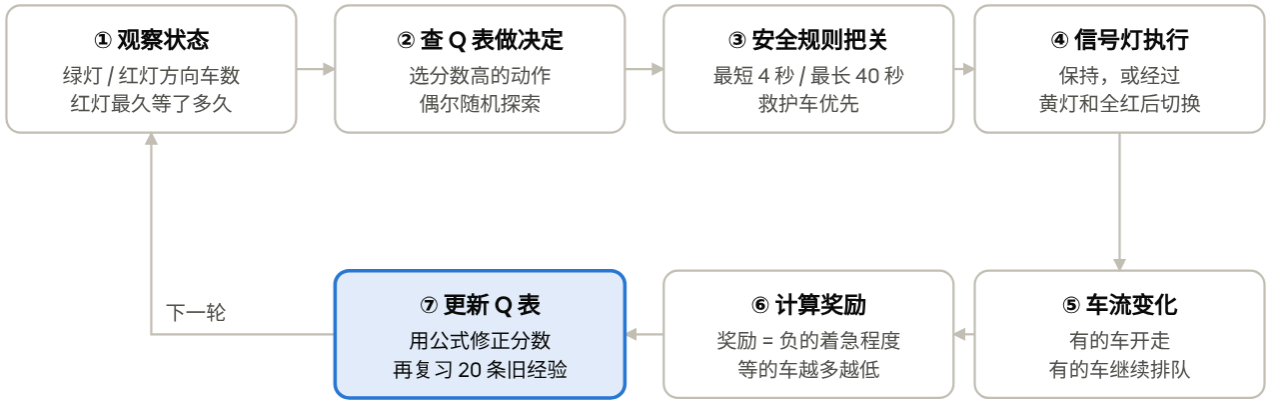
## 2. AI 大脑：Q 学习

| 概念 | 在本作品里是什么 |
| --- | --- |
| 状态 s | （绿灯方向车辆档位，红灯方向车辆档位，红灯方向最久等待档位），共 5×5×3 = 75 种路况 |
| 动作 a | 保持 / 切换 |
| 奖励 r | 负的“着急程度”：每辆在等的车 1 分，每多等 30 秒再加 1 分 |
| Q 表 | 每种路况下两个动作的分数，就是 AI 的“经验笔记本” |

每做完一个动作，AI 用下面的公式更新笔记本（α = 0.1 学习率，γ = 0.95 每秒折扣，t = 动作持续的秒数）：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \times ( r + \gamma^t \times \max Q(s', a') - Q(s, a) )$$

**每做一次决定，AI 都会看结果并更新 Q 表**




AI 学习循环 · 7 步，对应 controllers.py 和 brain.py

③ 是写死的规则，不参与学习；⑦ 是 AI 真正“变聪明”的地方。

## 3. 遇到的问题和解决办法

| 问题 | 解决办法 | 效果 |
| --- | --- | --- |
| 南北向学到的经验东西向用不上 | 状态改用“绿灯方向 / 红灯方向”的相对视角 | 状态数减半，学得快一倍 |
| AI 乱切换，黄灯 + 全红白白浪费 3 秒 | 用 γ^t 计算动作持续时间，让切换的真实代价进入奖励 | AI 学会了“不要乱切” |

| 问题 | 解决办法 | 效果 |
| --- | --- | --- |
| 只看排队数，AI 让车少的一边永远等 | 等得越久的车“着急程度”越高 + 最长绿灯 40 秒 | 不再有人被“饿死” |
| 同样训练 2 小时，有的 AI 聪明有的笨 | 经验回放：每学一条新经验，再随机复习 20 条旧经验 | 6 次独立训练全部学会好策略 |
| 新路况学得慢 | 自适应学习率：第一次遇到完全相信，见多了稳定在 0.1 | 清空大脑后约 4 分钟（仿真时间）反超定时灯 |
| 成绩虚高：只统计“已开走的车” | 改为“所有车一共等了多少秒 ÷ 来了多少辆车” | 被堵住的车也算进去，成绩真实 |

最后一条教会我：**选对评价指标和写对算法一样重要**。旧指标下 AI 看起来很强，实际上它会让一个方向的车永远走不了。

### 4. 安全规则层

AI 负责优化，规则负责兜底：安全相关的事不交给 AI 去“试错”。

- 最短绿灯 4 秒、黄灯 2 秒、全红 1 秒：写在信号灯里，AI 只能“申请切换”，想违规也做不到。
- 最长绿灯 40 秒：另一边有车时必须轮换。
- 救护车优先：发现救护车在红灯方向，立即为它开绿灯。

### 5. 一套代码，多端运行

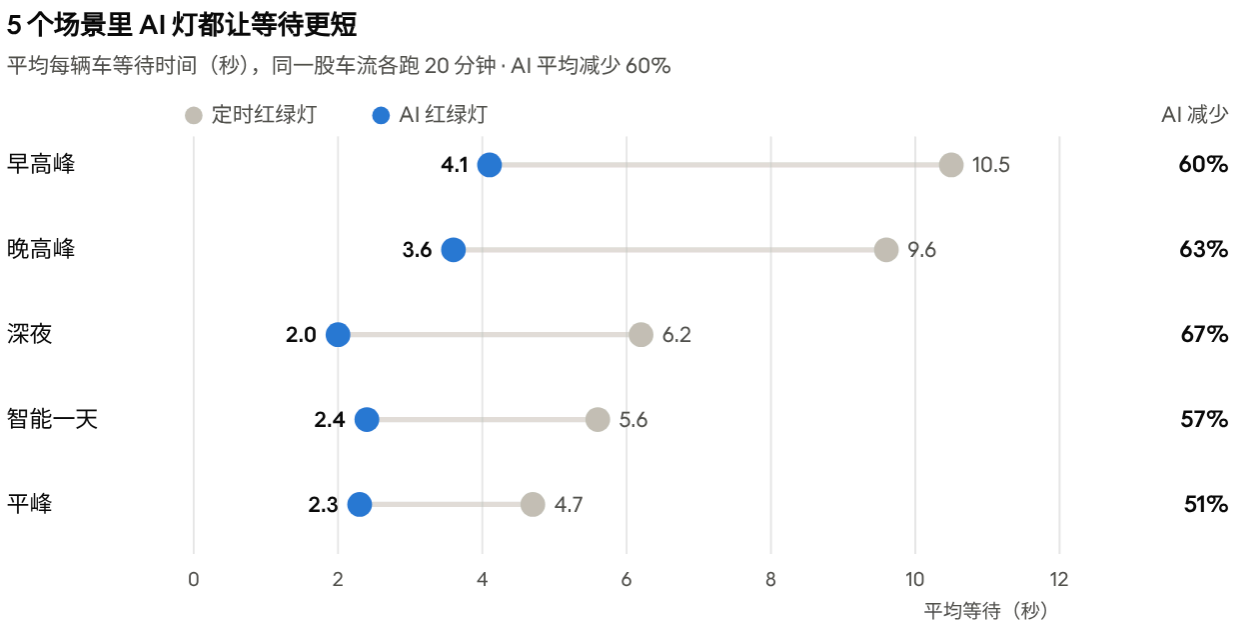
绘图代码不直接调用 tkinter，而是先生成一个“绘图指令清单”，比如 `["r", x, y, 宽, 高, 颜色]` 表示画一个矩形。电脑版用 tkinter 画清单，网页版用 Pyodide 在浏览器里运行同一份 Python，再用 canvas 画清单；演示视频也是用这份清单逐帧生成的。

### 6. 测试和调试

- `train.py`：不开画面，十几秒跑完 24 小时的训练，并打印“考试成绩表”。
- 9 个自动测试：不追尾、不闯红灯、AI 完全乱来时安全时间仍被遵守、Q 学习公式算得对、回忆库不会无限变大、AI 比定时灯好、救护车优先、界面能正常绘制。每次改代码后运行 `python -m unittest`。
- 所有参数集中在 `config.py`，做实验只改一个文件。

### 7. 实验成绩

同一股车流分别交给两种红绿灯，各跑 20 分钟仿真时间（关闭探索，只考真本事），AI 在 5 个场景里平均减少等待 60%，早高峰 20 分钟少排放约 3.2 千克 $CO_2$。运行 `python train.py --exam` 可以复现。




train.py --exam 输出 · 5 个场景，各 1200 秒仿真，随机种子 11

差距在早晚高峰最大：定时灯给两个方向同样的绿灯时间，车多的一边越排越长；深夜车少，AI “有车就放行”，减少得最多。

## 五、制作反思

最大的收获是明白了 AI 不是魔法：它的“聪明”来自看什么（状态）、追求什么（奖励）、怎么总结经验（学习算法）。

**收获。** 奖励设计错了，AI 就会“钻空子”：只看排队总数时，它会为了让数字好看而让少数车永远等着。要让 AI 做对的事，先要把“什么是对的”说清楚。

**难点。** AI 学习很不稳定，同样的代码换一个随机种子，结果就差很多。我用 6 个不同的种子反复做对比实验，才找到“经验回放”这个办法，让它又快又稳。

**不足和改进方向。**

- 现在只有直行，以后可以加入左转、行人过街。
- 现在只控制一个路口，以后可以让多个路口的 AI 互相配合，形成“绿波带”。
- Q 表只能处理分档后的路况，以后可以学习用神经网络（深度强化学习）处理更细致的路况。
- 仿真里的车流是模拟的，如果能接入真实交通数据，就能更接近实际应用。

**对 AI 伦理的思考。** 让 AI 管理交通时，效率不是唯一目标，还要考虑公平（不能让少数人一直等）、安全（规则兜底）和特殊需要（救护车优先）。

# 附录

## 评分标准对照

| 评分项 | 本作品的对应 |
| --- | --- |
| 思想性：契合主题，内容健康 | 用人工智能解决身边的交通拥堵和碳排放问题，关注公平与生命通道 |
| 规范性：说明文档、源码注释 | 本文档 + README；每个文件开头有中文说明，关键代码有注释 |
| 功能有效、成品化、人机交互 | 一键运行；按钮 + 快捷键 + 触屏；开场引导、原理说明浮层；网页和手机在线体验 |
| 创新性 | 双路口控制变量对比；Q 表可视化 + AI 心声；“AI 做梦”训练；一套代码多端运行 |
| 艺术性 | 夜景城市俯视风格；刹车灯、救护车警灯、等太久的“生气”提示等细节，美观与信息展示结合 |
| 技术性：结构合理、代码优化、易于调试 | 仿真 / AI / 控制 / 绘图 / 界面分层；纯手写 Q 学习 + 经验回放；自动测试与命令行训练脚本 |

## 演示视频

视频文件 `作品介绍视频.mp4` 在代码仓库里：3 分 56 秒，1920×1080，17 MB，带中文配音和字幕（字幕另存为 `作品介绍视频字幕.srt`），在线演示网页上也能直接播放。视频由 `python tools/make_video.py` 自动生成：画面是程序真实运行的每一帧，用的是和电脑版、网页版同一份绘图指令清单。

| 时间 | 画面 | 讲解要点 |
| --- | --- | --- |
| 0:00 | 标题 | 作者自我介绍、作品名称和一句话介绍 |
| 0:16 | 问题卡片 | 定时红绿灯不看路况；龙华早晚高峰车流方向相反 |
| 0:39 | 想法卡片 | 观察 → 尝试 → 看结果 → 总结经验 = 强化学习 |
| 0:51 | 程序画面 | 左边定时灯、右边 AI 灯，同一股车流 |
| 1:09 | 8 倍速早高峰 | 左边越排越长，右边轻松应对；看曲线和对比卡 |
| 1:43 | 派救护车 | AI 路口马上让路，定时路口干等 |
| 2:05 | 原理卡片 | 观察、决定、打分、学习四步 + Q 表更新公式 |

| 时间 | 画面 | 讲解要点 |
| --- | --- | --- |
| 2:38 | 清空大脑 → 极速训练 | Q 表从“？”长出规律，AI 反超定时灯 |
| 3:05 | 实验成绩 | 5 个场景全部获胜，平均减少等待 60% |
| 3:21 | 技术亮点 | 0 第三方库、一套代码四种画法、9 个自动测试 |
| 3:43 | 总结展望 | 多路口绿波带、行人和左转、真实交通数据 |

## 运行方法

1. 电脑：用海龟编辑器或 IDLE 打开 `main.py`，点“运行”。保持 `main.py` 和 `smartlight` 文件夹在同一个目录，不需要安装任何库。
2. 网页 / 手机：打开 zongruichd-rd.github.io/bisai2，手机上点“全屏演示”。
3. 命令行：`python train.py`（训练 + 考试）、`python -m unittest`（自动测试）。